Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Dersin tarihi 14 Nisan 2021 Çarşamba. Ders Zoom üzerinden yapıldı.
  - İlk yaklaşık 1 saat 18 dakikayı Ahmet Elbir anlatır. Kalan bölüm Toprak Hoca'nın (asistan) uygulamasıdır.
  - 1:05 ile 1:09 arasında hocanın bağlantısı koptu. Bu arada Toprak Hoca kısa bir bekleme anonsu yaptı.
- Duyurular
  - Ramazan nedeniyle laboratuvar saat 21:00'de başlayacak ve yaklaşık 22:20-22:30'a kadar sürecek. Amaç, öğrencilerin aç ve yorgun hâlde sınava girmemesi.
  - Bu hafta laboratuvar yok, uygulama var. Gelecek hafta laboratuvar var.
  - 12 Mayıs'taki laboratuvar arefe gününe denk geliyor. Resmî tatilde Zoom oturumu tanımlanmadığı için üç grup için ortak bir Zoom oturumu açılacak. Kodlar yine sisteme yüklenecek.
  - Hoca, diğer grupların (Yunus Hoca, Mehmet Hoca) geride olduğunu, bu yüzden bu hafta çok yeni konu açmayacağını söyledi.
- Tekrar: Nesneler arası ilişkiler
  - Toplam 5 ilişki türü var. Kalıtım (inheritance) sınıf seviyesindedir. Diğer 4'ü nesne seviyesindedir.
  - **Association ("has"):** Bir nesne başka bir nesnenin referansını saklar. Örnek: rafta kitap, kişinin arabası. Sayı 0 veya çok olabilir.
  - **Dependency:** Bir nesne başka bir nesneyi yalnızca bir metodunun içinde kullanır. UML'de kesik çizgiyle gösterilir.
  - **Aggregation ve composition ("whole-part"):** Bütün-parça ilişkisi. Ders kapsamında en az 1 parça olmalıdır. Örnek: otobüs hattı ve duraklar.
    - Aggregation (içi boş elmas): Parça nesnesi dışarıda üretilip gönderilir. Bütün silinince parça yaşamaya devam eder. Hat silinince duraklar silinmez.
    - Composition (içi dolu elmas): Parça, bütünün içinde üretilir. Bütün yok olunca parça da yok olur. Örnek: canlı ve organları.
  - **Kalıtım ("is-a"):** B, A'nın özelliklerini alıyorsa "B bir A'dır". UML'de üçgen uçlu okla gösterilir. Örnek: öğrenci bir kişidir.
- Tekrar: Overload ve override
  - **Overload:** Dönüş tipi ve metot adı aynı kalır, parametre sayısı veya tipleri değişir. Kalıtıma özel değildir. Aynı sınıfta da yapılır.
  - **Override:** Üst sınıftaki bir metodu alt sınıfta aynı imzayla yeniden yazmaktır. Ya tamamen geçersiz kılar ya da `super.metot()` ile üst sınıftakini çağırıp eklenti yapar. Kalıtımla ilgilidir. Çok biçimliliğin (polymorphism) temelidir.
- Soyut sınıflar (abstract classes)
  - `Person` sınıfındaki boş `yaz()` ve `bakiyeGuncelle()` metotları örneğiyle başlandı.
  - Hedef: Bu metotlar üst sınıfta yalnızca tanımlansın, gövdesi olmasın. Bu sınıftan kalıtım alan her sınıf bu metotları yazmak zorunda olsun.
  - Çözüm: `public abstract void yaz();` Gövde yazılmaz, satır noktalı virgülle biter.
  - Soyut sınıf bir şablondur (template). Somut sınıfın (concrete class) karşıtıdır.
- Soyut sınıflarla ilgili İngilizce slayt anlatımı
  - Soyut sınıfın alanları, constructor'ı, somut ve soyut metotları olabilir.
  - Kalıtım hiyerarşisinde yukarı çıktıkça sınıflar genelleşir. Bir noktadan sonra nesne üretmeye gerek kalmaz. Örnek: `Object` sınıfından doğrudan nesne üretmeyiz. Görsel

programlamada `JFrame` de kalıtım yoluyla kullanılır.
  - UML'de soyut sınıf ve metot italik yazılır veya `<<abstract>>` stereotipi eklenir. İtalik bazen görünmediği için stereotip daha güvenlidir.
  - `PrintDriver` örneği: Her sürücünün `print` metodu vardır ama her sürücü farklı yazdırır. `print` soyut tanımlanır.
  - **Dinamik bağlama (dynamic binding):** Referans `PrintDriver` türünde olsa da çalışma zamanında gerçek nesnenin (`PSDriver`, `PCL6Driver`) metodu çağrılır.
- Çoklu kalıtım (multiple inheritance)
  - Örnek: Asistan öğrenci hem öğrenci hem çalışandır. `class AssistantStudent extends Student, Employee` yazılamaz.
  - Java ve C# sınıflarda çoklu kalıtımı desteklemez. Sebep: elmas problemi (diamond problem). İki üst sınıf aynı `toString`'i farklı override ederse alt sınıfta hangisinin çalışacağı belirsiz olur.
  - Python çoklu kalıtımı destekler.
- Tasarım örneği: Kreş, çocuk, eşya (`Item`, `Toy`, `Clothing`)
  - `Item` soyut sınıftır. `isSuitable` soyut metottur. Oyuncaklarda alt yaş sınırı (yıl), giysilerde alt ve üst yaş sınırı (ay) vardır.
  - Sınıfları yazma sırası önemlidir: önce `Child` (dependency ile kullanılıyor), sonra `Kindergarten`, sonra `Item`, en son `Toy` ve `Clothing`. Rastgele sırada yazarsanız çok sayıda derleme hatası alırsınız.
  - `Toy` constructor'ı, `Item`'e ait `barcode` ve `description` bilgilerini `super(...)` ile göndermek zorundadır. Bu sınavda da soruldu.
- Kalıtımda nesne oluşturma sırası (`NoWait`, `Steve`, `Alan` örneği)
- Overload çözümleme örneği (`Se7en` sınıfı, `sin` metotları)
- Toprak Hoca'nın uygulaması: Kitaplık sistemine soyut `Kart` sınıfı eklenmesi
  - `kitapSil` metotları aynı ada getirildi (overload). Biri kitap nesnesi, diğeri raf numarası alıyor.
  - `Kitap` sınıfına `fiyat` alanı ve getter eklendi. `kitapAra` artık kitabın bulunduğu raf numarasını, bulamazsa `-1` döndürüyor.
  - Soyut `Kart` sınıfı: `id`, `sahibi`, `bakiye` alanları; soyut `boolean odemeYap(double fiyat)` metodu.
  - `YetiskinKart extends Kart`: Bakiye yetiyorsa bakiyeden düşer, `true` döner. Yetmiyorsa `false` döner.
  - `OgrenciKart extends Kart`: Ek olarak `bonus` alanı var. Bakiye yetmezse bonusa bakar. Bakiyeden ödeme yapınca fiyatın %20'si bonusa eklenir.
  - `OgrenciKart` kendi `toString` metodunu yazar ve `super.toString()` ile üst sınıfın metnine bonus bilgisini ekler (override örneği).
  - `KitaplikYonetim` sınıfı: `kitapSat(Kitap kitap, Kart kart)` metodu. Kitap yoksa `false`. Varsa `kart.odemeYap(kitap.getFiyat())` çağrılır.
  - Polimorfizm: Kartlar `Kart` türünde referansla da oluşturulabilir. `KitaplikYonetim` hangi kart olduğunu bilmeden çalışır. Yeni bir kart türü eklenince bu sınıf hiç değişmez.

Hocanın Özellikle Vurguladığı Kısımlar

- **Bir sınıf en az bir soyut metot içeriyorsa soyut tanımlanmak zorundadır. Ama soyut bir sınıfın soyut metot içermesi zorunlu değildir.**
  - Hoca bunun tersinin ("sınıf soyutsa soyut metot içerir") yanlış olduğunu birkaç kez vurguladı.
- Soyut sınıflar iki sonuç doğurur.
  - Kısıtlama: Soyut sınıftan nesne üretilemez. Yalnızca referans üretilebilir: `Person p = new Student(...);`
  - Zorunluluk: Kalıtım alan somut sınıf, bütün soyut metotları implement etmek zorundadır. Etmezse derleme hatası alınır.
- Soyut bir metodu alt sınıftan `super.yaz()` ile çağıramazsınız. Gövdesi yoktur. C'deki fonksiyon prototipi gibi düşünün.
- Soyut metot bir standart belirler. Örnek: Araba üreten herkes gaz pedalı ve vites koymak zorundadır.
- **Kalıtımda nesne oluşturma sırası: önce statikler (en üst sınıftan başlayarak), sonra üye alanlar (member fields), en son constructor.**

  - “Nesne üretilince constructor çalışır” demek tam doğru değil. Constructor’dan önce sınıfın alanları oluşturulmalıdır.
- Bir statik blok veya statik alan yalnızca bir kez çalışır.
- Kavramları tanımıyla bilin. Örnek üzerinden tanım yapmayın.
  - “Override nedir?” sorusuna “toString’i değiştirmek” demek yeterli değil. Hoca staj ve iş mülakatlarında polymorphism, abstract ve inheritance sorulduğunu, cevaplayamayanın “nesne yönelimli programlamayı bilmiyor” sayıldığını söyledi.
- Overload kalıtıma özel değil, override ise kalıtımla ilgilidir. Override, polimorfizme yol açar.
- Overload çözümlemede küçük tipten büyük tipe dönüşüm otomatik yapılır. Büyükten küçüğe (kayıplı) dönüşüme izin verilmez.

Kısa Tekrar Notları

- 5 ilişki: kalıtım (“is-a”, sınıf seviyesi), association (“has”), dependency (kesik çizgi), aggregation (içi boş elmas, parça yaşar), composition (içi dolu elmas, parça ölür).
- Overload: aynı ad, farklı parametre listesi. Override: alt sınıfta aynı imza.
- `abstract` metot: gövdesi yok, `;` ile biter.
- Soyut metot varsa sınıf soyut olmalı. Soyut sınıfın soyut metodu olmayabilir.
- Soyut sınıftan `new` ile nesne üretilmez. Referans üretilir.
- Somut alt sınıf bütün soyut metotları yazmak zorunda.
- Dinamik bağlama: referans türü değil, gerçek nesnenin türü hangi metodun çalışacağını belirler.
- Java’da sınıflar arası çoklu kalıtım yok (elmas problemi). Python’da var.
- Nesne oluşturma sırası: statikler (hiyerarşide yukarıdan aşağı), üye alanlar, constructor.
- `this(...)` aynı sınıftaki başka constructor’ı, `super(...)` üst sınıfın constructor’ını çağırır.

Detaylı Açıklamalar

Soyut sınıf ve soyut metot

```
public abstract class Person {
    private int id, yas;
    private double bakiye;
    public Person(int id, int yas) { this.id = id; this.yas = yas; }
    public void yasArttir() { yas++; }          // somut metot, alt sınıflar aynen kullanır
    public abstract void yaz();                  // soyut metot
    public abstract double bakiyeGuncelle();     // soyut metot
}

public class Student extends Person {
    public Student(int id, int yas) { super(id, yas); }
    @Override
    public void yaz() { System.out.println("Ogrenci yaz"); }
    @Override
    public double bakiyeGuncelle() { /* öğrenciye özgü hesap */ return 0.0; }
}
```

- `Person p = new Person(...)` artık derleme hatası verir. `Person p = new Student(...)` geçerlidir.
- `yasArttir()` gibi somut metotları alt sınıflar override etmek zorunda değildir. Override etmezlerse üst sınıftaki çalışır.
- Bir öğrenci “Person’daki soyut olmayan metotlar bir daha kullanılmaz mı?” diye sordu. Hoca bunun yanlış olduğunu söyledi. Somut metotlar alt sınıflarda aynen kullanılır.

Kalıtımda nesne oluşturma sırası

Derste ekran üzerinde çalıştırılan kod (kareden alındı):

```
class NoWait {
    public NoWait() { System.out.println("NoWait Konstruktor"); }
```

```
}

class Steve extends NoWait {
    static { System.out.println("Steve statischer Initialisierer"); }
    public Steve() { this(2); System.out.println("Steve Konstruktor 1"); }
    public Steve(int i) { super(); System.out.println("Steve Konstruktor 2"); }
}

public class Alan extends Steve {
    static NoWait alan = new NoWait();
    public Steve murmeltier = new Steve(5);
    public Alan() { System.out.println("Alan Konstruktor"); }
    public static void main(String[] args) { new Alan(); }
}
```

Çıktı (Java ile çalıştırılıp doğrulandı):

```
Steve statischer Initialisierer
NoWait Konstruktor
NoWait Konstruktor
Steve Konstruktor 2
Steve Konstruktor 1
NoWait Konstruktor
Steve Konstruktor 2
Alan Konstruktor
```

Adım adım:

1. Sınıflar yüklenirken statikler hiyerarşide yukarıdan aşağı çalışır. `NoWait`'te statik yok. `Steve`'in statik bloğu "Steve statischer Initialisierer" yazar.
2. Sonra `Alan`'ın statik alanı `alan = new NoWait()` çalışır: "NoWait Konstruktor".
3. `new Alan()`: `Alan()` önce örtük olarak `super()` yani `Steve()` çağırır. `Steve()` içindeki `this(2)` ile `Steve(int)` çağrılır. O da `super()` ile `NoWait()` çağırır: "NoWait Konstruktor", "Steve Konstruktor 2", "Steve Konstruktor 1".
4. Üst sınıf kısmı bitince `Alan`'ın üye alanı `murmeltier = new Steve(5)` oluşturulur: "NoWait Konstruktor", "Steve Konstruktor 2".
5. En son `Alan()`'ın gövdesi: "Alan Konstruktor".

- Statik blok ikinci kez çalışmaz. `Steve` nesneleri tekrar üretilse de "Steve statischer Initialisierer" bir kez görünür.
- Üye alanın sınıfta nerede yazıldığı fark etmez. Üye alanlar her zaman constructor gövdesinden önce oluşturulur.

Overload çözümleme (`Se7en` örneği)

```
public class Se7en {
    public static void sin(int x, char y, int z)     { System.out.print("Bir "); }
    public static void sin(int x, short y, long z)   { System.out.print("İki "); }
    public static void sin(char x, float y, double z){ System.out.print("Üç "); }
    public static void sin(int x, int y, float z)    { System.out.print("Dört "); }

    public static void main(String[] args) {
        char c = '9'; short s = 8; int i = 7; long l = 6; float f = 5; double d = 4;
        sin(c, f, d);     // Üç   (birebir eşleşme)
        sin(i, c + s, i); // Dört (c + s int olur; int,int,int -> int,int,float)
        sin(i, c, i);     // Bir  (birebir eşleşme)
        sin(i, i, f);     // Dört
        sin(s + s, s, s); // İki  (s + s int olur; short -> long genişler)
        sin(s, s, s);     // İki
        // sin(i, l, s);  // derleme hatası: long -> char/short/int daraltılamaz
        // sin(c, i, i);  // derleme hatası: ambiguous (Üç ve Dört ikisi de uyar)
    }
```

```
}
```

- Derleyici önce birebir eşleşen metodu arar. Yoksa genişletici dönüşümle (widening) uyan metotlara bakar ve en özel (en yakın) olanı seçer.
- Büyük tipten küçük tipe dönüşüm (ör. `long` -> `char`) otomatik yapılmaz. Uyan metot yoksa "no suitable method found" hatası alınır.
- `sin(c, i, i)` için hem `(char, float, double)` hem `(int, int, float)` uyar ve biri diğerinden daha özel değildir. Derleyici "reference to sin is ambiguous" hatası verir.

Toprak Hoca'nın kart örneği

```
public abstract class Kart {
    private int id; private String sahibi; private double bakiye;
    public Kart(int id, String sahibi, double bakiye) { ... }
    public abstract boolean odemeYap(double fiyat);
    // get/set ve toString
}

public class OgrenciKart extends Kart {
    private double bonus = 0;
    public OgrenciKart(int id, String sahibi, double bakiye) { super(id, sahibi, bakiye); }
    @Override
    public boolean odemeYap(double fiyat) {
        if (getBakiye() >= fiyat) {
            setBakiye(getBakiye() - fiyat);
            bonus += fiyat * 0.2;
            return true;
        } else if (bonus >= fiyat) {
            bonus -= fiyat;
            return true;
        }
        return false;
    }
    @Override
    public String toString() { return super.toString() + " bonus=" + bonus; }
}
```

- 50 TL bakiyeli öğrenci kartı 30 TL ödeyince bakiye 20 TL, bonus 6 TL olur.
- 10 TL'lik kitaplar: Yetişkin kartı (50 TL) 5 kitap alır. Öğrenci kartı 5 kitaptan sonra biriken 10 TL bonusla bir kitap daha alır.

Ders sonrası notlar

- **`c + s` işleminin tipi:** Hoca "karakterle short toplanınca short olur" dedi. Java'da `char + short` işleminde iki işlenen de `int`'e yükseltilir (binary numeric promotion). Sonuç `int`'tir. `sin(i, c + s, i)` bu yüzden `(int, int, float)` sürümüne, yani "Dört"e gider. Sonuç aynı ama gerekçe farklı.
- **"Overwrite" değil "override":** Derste ve uygulamada sık sık "overwrite" dendi. Doğru terim override'dır (`@Override`).
- **Bütün-parça ilişkisinde "en az bir" kuralı:** Bu, dersin kullandığı yorumdur. UML'de aggregation ve composition için çokluk (multiplicity) `0..*` da yazılabilir. Asıl ayırt edici nokta parçanın yaşam süresidir.
- **Çoklu kalıtım:** Java ve C# sınıflar arasında çoklu kalıtımı desteklemez. Ama bir sınıf birden fazla interface'i `implements` edebilir. C++ ve Python sınıflar arası çoklu kalıtımı destekler.
- **Interface'lerdeki default metotlar:** Hoca "yeni Java sürümlerinde interface'e default metot yazılabilir" dedi. Bu özellik Java 8 ile geldi.

- **`sin(i, c + s, i)` çalışırken hoca "4 nasıl çalıştı?" diye şaşırdı.** Yukarıdaki not bunun nedenini açıklar. Bütün çağrılar Java ile çalıştırılıp doğrulandı.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.